

Modern Day Computing System: A 16-Bit Computer Built from First Principles

A Comprehensive Technical IEEE-Format Project Report

Author: Dhruv Sud(102395017) and Ishan Chaturvedi(102256006)

Under the mentorship of Dr. Sushil Jain

Project: MODERN DAY COMPUTING SYSTEM

License: MIT Open Source License | © 2026

Repository: <https://modern-day-computing-system.github.io/PROJECT/>



**DEPARTMENT OF ELECTRONICS AND COMMUNICATIONS
THAPAR INSTITUTE OF TECHNOLOGY AND ENGINEERING,
(DEEMED TO BE UNIVERSITY), PATIALA, PUNJAB
JANUARY – MAY, 2026**

ABSTRACT

This paper presents a comprehensive design and implementation report for the "Modern Day Computing System" project — a complete 16-bit computer architecture constructed entirely from first principles. The system encompasses all classical layers of a computing stack: combinational logic gates, an Arithmetic Logic Unit (ALU), CPU registers and instruction decoding, hierarchical RAM and ROM memory, memory-mapped display and keyboard I/O, and an accompanying custom assembler. All digital circuits were designed and simulated using Logisim-Evolution v3.8.0 with circuit files stored in Logisim's XML-based .circ format. The assembler was implemented in Java and translates a custom assembly language (inspired by the Hack machine language from Nand2Tetris) into 16-bit binary machine code, which is then converted to hexadecimal for ROM loading. A web-based RGB555 Bitmap Editor (BitmapEditor.html) provides a pixel-art design tool that auto-generates assembly code for framebuffer rendering. The system also includes a suite of assembly language programs demonstrating arithmetic operations, Fibonacci computation, keyboard polling, bitmap rendering, pointer manipulation, and 3D shape rendering. This report documents every file, design decision, architectural component, and software implementation in full, constituting a complete technical reference for the project. The work draws inspiration from Nand2Tetris, classical digital computer architecture, and the Logisim ecosystem.

Keywords: 16-bit architecture, ALU design, CPU simulation, Harvard Architecture, Logisim-Evolution, custom assembler, Java, assembly language, memory-mapped I/O, framebuffer, RGB555, Nand2Tetris, digital logic.

TABLE OF CONTENTS

I.	Introduction and Project Overview	4
II.	System Architecture	5
III.	Arithmetic Logic Unit (ALU)	6
IV.	Central Processing Unit (CPU)	9
V.	Memory Architecture	13
VI.	Java Assembler — Full Implementation	17
VII.	Assembly Language Programs	21
VIII.	Display and I/O Subsystems	25
IX.	RGB555 Bitmap Editor Tool	27
X.	Project Infrastructure, Git History, and Build System	29
XI.	Conclusion and Future Work	30
XII.	References	30

I. Introduction and Project Overview

The "Modern Day Computing System" is an educational and community-oriented project that builds a complete 16-bit digital computer from the ground up, using nothing but primitive logic gates as the foundational building blocks. The project is motivated by the belief that true understanding of computing requires intimate knowledge of what happens at every layer of the system stack — from electrons flowing through transistor gates all the way to executable programs running in memory.

The project takes heavy inspiration from "The Elements of Computing Systems" by Nisan and Schocken (Nand2Tetris) and from Albert Paul Malvino's "Digital Computer Electronics," both of which are included in the repository's Books/ directory. However, this implementation goes beyond the textbook exercises and incorporates original design decisions, custom tooling, and extended experiments in areas such as synchronization, RGB framebuffer graphics, and 3D shape rendering in assembly.

The repository is hosted on GitHub at <https://modern-day-computing-system.github.io/PROJECT/> and is licensed under the MIT License (Copyright 2026 MODERN DAY COMPUTING SYSTEM). The project is an active work-in-progress, as indicated by the "Work In Progress" badge in the README and the ongoing git commit history spanning from March 2026 to May 2026.

I-A. Project Goals

- Build a fully functional 16-bit computer architecture from primitive logic components.
- Understand instruction execution at the hardware level — from fetch to decode to execute.
- Design memory systems (RAM, ROM, registers) from scratch in Logisim-Evolution.
- Implement a custom Java-based assembler that converts human-readable assembly into binary machine code.
- Explore framebuffer and display systems using memory-mapped I/O.
- Experiment with keyboard I/O and hardware-software interaction.
- Lay the groundwork for future operating system development and compiler backend exploration.
- Build a web-based RGB555 Bitmap Editor that generates assembly code from pixel artwork.

I-B. Repository Structure

The project repository contains the following top-level directories and files:

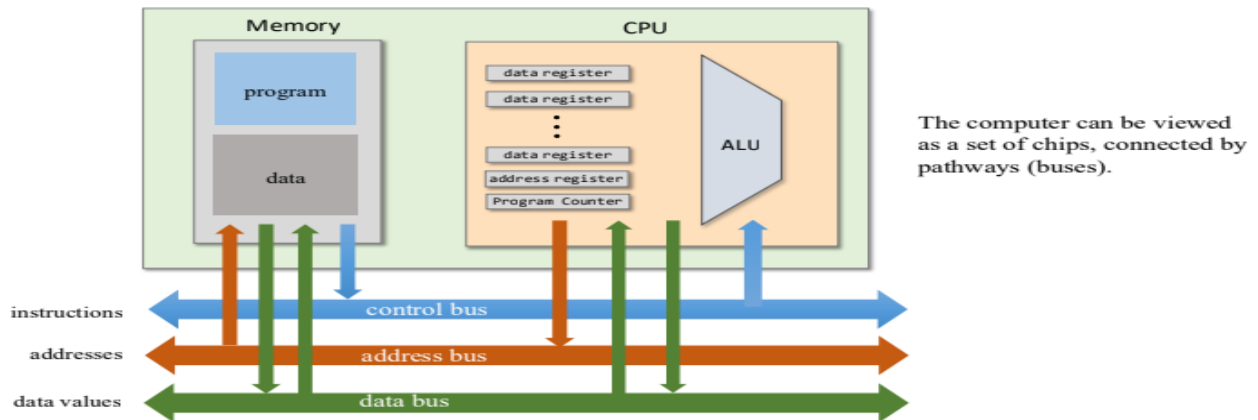
Directory / File	Description
ALU/	Logisim circuit files and PNG images for the Arithmetic Logic Unit (ALU), 16-bit adder, and 16-bit multiplexer.
Assembler/	Java source files for the custom assembler (Assembler.java), hexadecimal converter (HexadecimalConverter.java), and test script (TestScript.java), along with compiled .class files, input (Code.asm), and output files (output.hack, hex.txt, HackAssemblerOutput.hack).
assembly_programs/	Assembly language programs: fibonacci.asm, MAX.asm, keyboard.asm, bitmap.asm, RGB.asm, pointer_array.asm, fibonacci_pointer_array_sum.asm, iterative_addition.asm, rom_execution.asm, Soldier.asm (5208 lines), 3DShapes.asm (2768 lines).
Backup/	Backup Logisim circuits for Keyboard.circ and Screen.circ subsystems.
Books/	Reference PDFs: "The Elements of Computing Systems" and "Digital Computer Electronics."
COMPUTER/	Complete assembled computer model: Model1.circ and Model1.png (4396x1321 px).
CPU/	CPU Logisim circuit (CPU.circ), circuit images (CPU.png, Colored.png), hardware debugger photo, and Karnaugh map for jump logic (kmapJUMPsolution.pdf).

Course Work/	Academic course poster (Poster.png, 2245x1587 px).
Keyboard/	Keyboard subsystem image (Keyboard.png).
MEMORY/	RAM circuit files (8RAM.circ, RAM32kB.circ), memory images, program counter PNG, register PNG, synchronization experiment images.
PixelCommunitySubmissions /	Community-submitted pixel art screenshots from the Bitmap Editor collaborative feature.
Screen/	Screen subsystem image (Screen.png).
BitmapEditor.html	698-line single-file web application implementing an RGB555 pixel art editor with live assembly code generation.
README.md	Project overview, architecture diagrams, usage instructions, and component descriptions (10,070 bytes).
LICENSE	MIT License, Copyright 2026.
references.txt	References of all the brilliant people who help make this possible.

II. System Architecture

The Modern Day Computing System implements a Harvard architecture adapted from the Hack computer specification. The complete system integrates an instruction memory (ROM), a data memory (RAM), a CPU with internal registers and ALU, and memory-mapped I/O regions for screen and keyboard access. Figure 1 shows the complete assembled computer model as simulated in Logisim-Evolution.

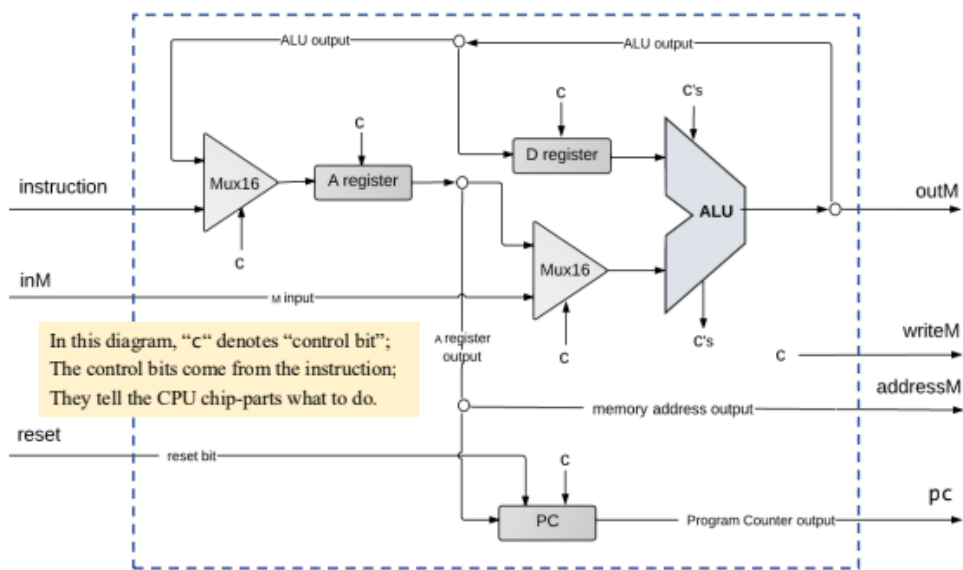
Basic architecture



Nand to Tetris / www.nand2tetris.org / Chapter 5 / Copyright © Noam Nisan and Shimon Schocken

Slide 18

CPU implementation



Hack CPU architecture

Nand to Tetris / www.nand2tetris.org / Chapter 5 / Copyright © Noam Nisan and Shimon Schocken

Slide 37

Harvard Architecture

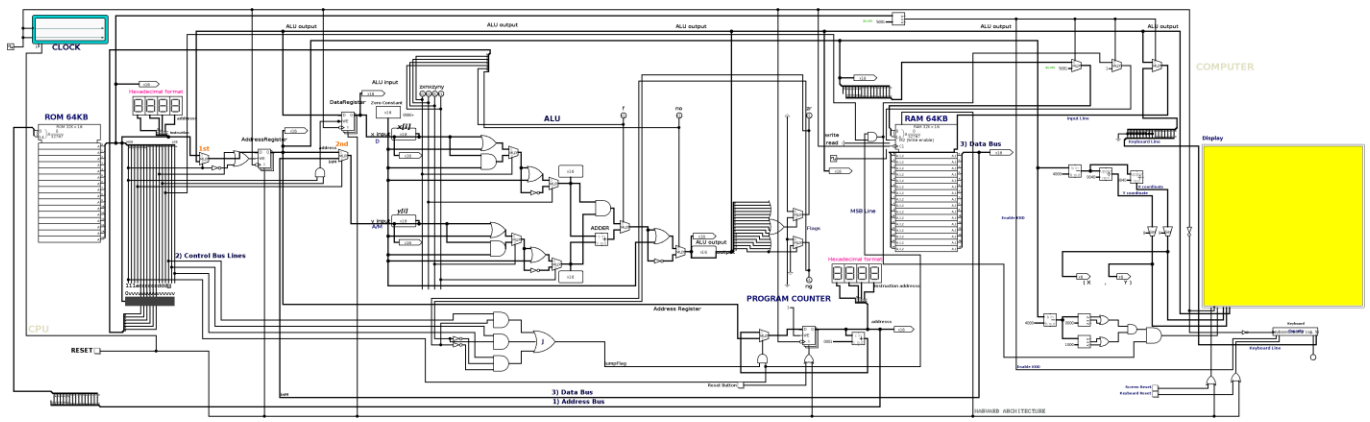


Fig. 1. Complete Computer Model (Model1.circ) — full system integration in Logisim-Evolution. The circuit spans a 4396x1321 pixel canvas and shows the CPU, ROM, RAM, and I/O connections.

II-A. Instruction Set Architecture

The system employs a 16-bit fixed-width instruction set with two instruction types: A-instructions and C-instructions. This binary encoding is directly inspired by the Hack machine language.

A-Instruction (Address Instruction)

Format: 0vvvvvvvvvvvvvvv (bit 15 = 0, bits 14:0 = 15-bit address/value)

Purpose: Loads a 15-bit non-negative integer constant into the A-register, or sets the A-register to point to a memory address. The assembler generates this for @<number> and @<label> mnemonics.

C-Instruction (Compute Instruction)

Format: 111accccccddjjj (bits 15:13 = 111, bit 12 = a-bit, bits 11:6 = comp, bits 5:3 = dest, bits 2:0 = jump)

The a-bit selects whether the ALU operates on the A-register (a=0) or on Memory[A] (a=1). The 6-bit comp field encodes the ALU operation. The 3-bit dest field specifies destination register(s) (A, D, M, combinations). The 3-bit jump field specifies conditional or unconditional jumps.

II-B. Register Set

The CPU contains three user-accessible registers:

- A-Register (16-bit): Holds a data value or a memory address. Used both as a data source and for addressing.
- D-Register (16-bit): General-purpose data register. The ALU always operates between D and either A or M.
- M (Memory[A]): Not a physical register — a shorthand for the RAM word at the address held in A.

II-C. Memory Map

The 16-bit address space is divided as follows:

Address Range	Size	Purpose
0 – 15	16 words	General-purpose registers R0–R15 (also SP, LCL, ARG, THIS, THAT)
16 – 16383	16368 words	User variable RAM space
16384 – 20480	4096 words	Screen memory-mapped I/O (framebuffer, 64x64 pixels, 1 word per pixel; extended to RGB555 in custom implementation)

20481 (KBD)	1 word	Keyboard memory-mapped I/O (ASCII code of currently pressed key, or 0 if none)
-------------	--------	--

The assembler pre-defines the symbolic constants SCREEN;16384, KBD;20481 (the project uses 20481 for the keyboard address in the predefined table), SP;0, LCL;1, ARG;2, THIS;3, THAT;4, and R0–R15.

III. Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit is the computational heart of the CPU. It is implemented in Logisim-Evolution as `ALU_circuits.circ` (406 lines of XML) and supports 18 distinct operations selectable by a 6-bit control word (`accccc`). The ALU takes two 16-bit inputs and produces a 16-bit output, along with status flags.

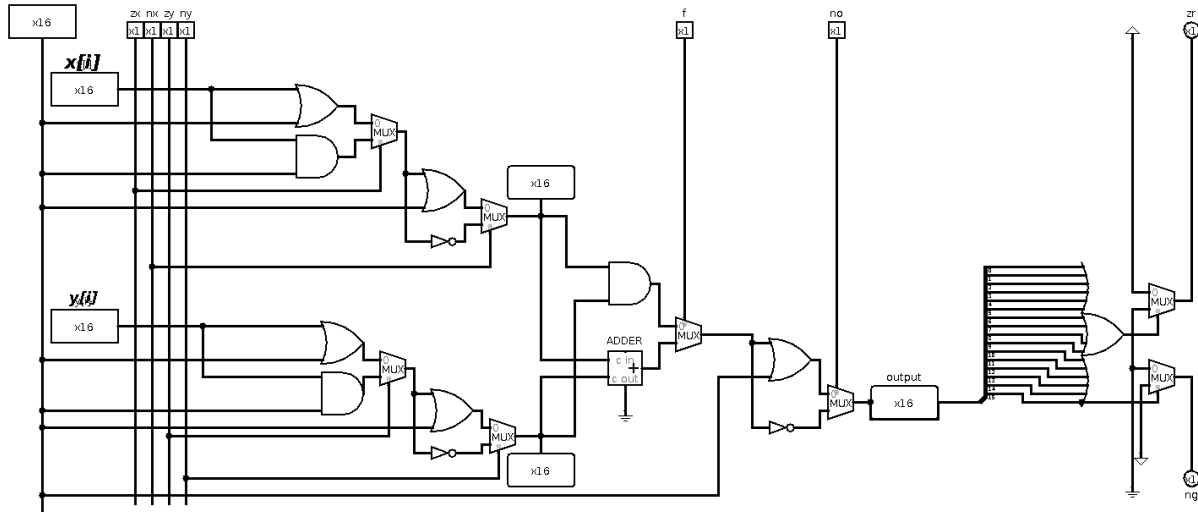


Fig. 2. ALU Circuit (`alu.png`) — the complete Arithmetic Logic Unit in Logisim-Evolution simulation view, showing 16-bit data paths, multiplexer chains, and control signal routing.

III-A. ALU Operation Table

The `ControlBits` inner class of the Assembler encodes the following ALU operations. The `a-bit` (bit 12 of the C-instruction) selects A-register (`a=0`) or `M=Memory[A]` (`a=1`) as the second operand. The following table, derived directly from the computation array in `ControlBits`, documents all supported computations:

Mnemonic	cccccc	a-bit	Description
0	101010	0	Output constant zero
1	111111	0	Output constant one
-1	111010	0	Output constant negative one (all 1s in two's complement)
D	001100	0	Pass D-register value through
A	110000	0	Pass A-register value through
!D	001101	0	Bitwise NOT of D
!A	110001	0	Bitwise NOT of A
-D	001111	0	Two's complement negation of D
-A	110011	0	Two's complement negation of A
D+1	011111	0	Increment D by 1
A+1	110111	0	Increment A by 1
D-1	001110	0	Decrement D by 1
A-1	110010	0	Decrement A by 1
D+A	000010	0	Add D and A (also accepts A+D)

D-A	010011	0	Subtract A from D
A-D	000111	0	Subtract D from A
D&A	000000	0	Bitwise AND of D and A (also accepts A&D)
D A	010101	0	Bitwise OR of D and A (also accepts A D)

When the a-bit is 1 (instruction contains memory reference M), the assembler substitutes M with A in the computation string before table lookup, and the CPU routes Memory[A] into the ALU's second input bus instead of the A-register.

III-B. 16-Bit Adder Circuit

The adder.circ file (Logisim source) implements the 16-bit ripple-carry adder used inside the ALU for D+A, D+1, A+1, D-1, A-1, and negation operations. It is composed of 16 cascaded full-adder stages. Figure 3 shows the adder circuit.

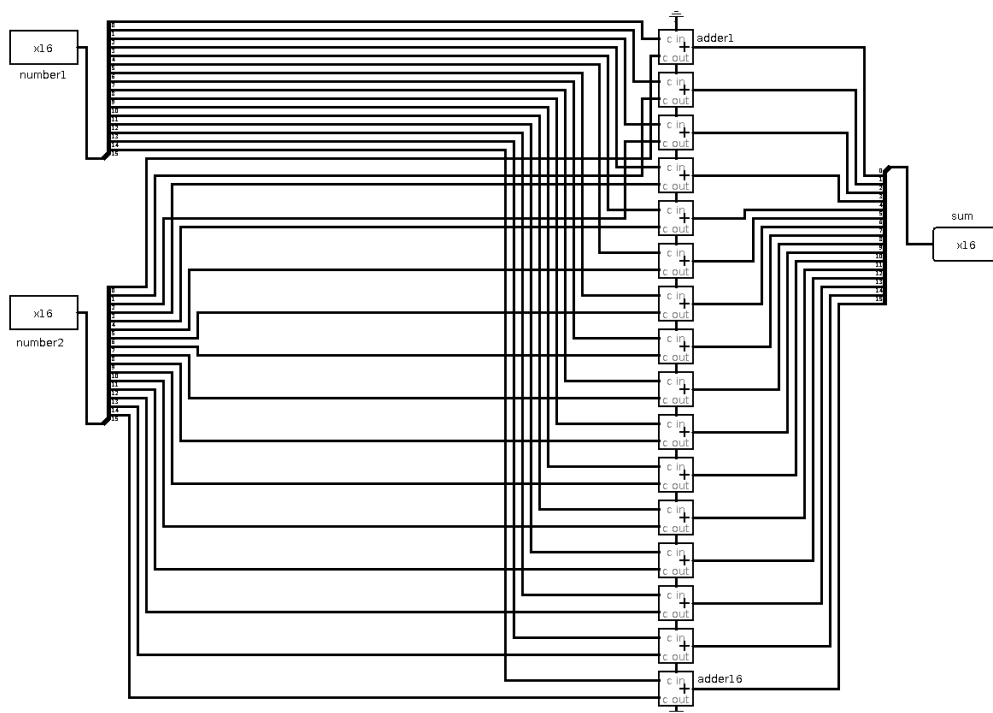


Fig. 3. 16-Bit Adder Circuit (adder.png) — hierarchical ripple-carry adder implemented in Logisim, showing 16 full-adder stages with carry propagation.

III-C. 16-Bit Multiplexer (mux16.circ)

The mux16.circ implements a 16-bit 2-to-1 multiplexer used throughout the ALU and CPU for data path selection. The circuit selects between two 16-bit input buses based on a single control bit. Figure 4 shows the multiplexer circuit.

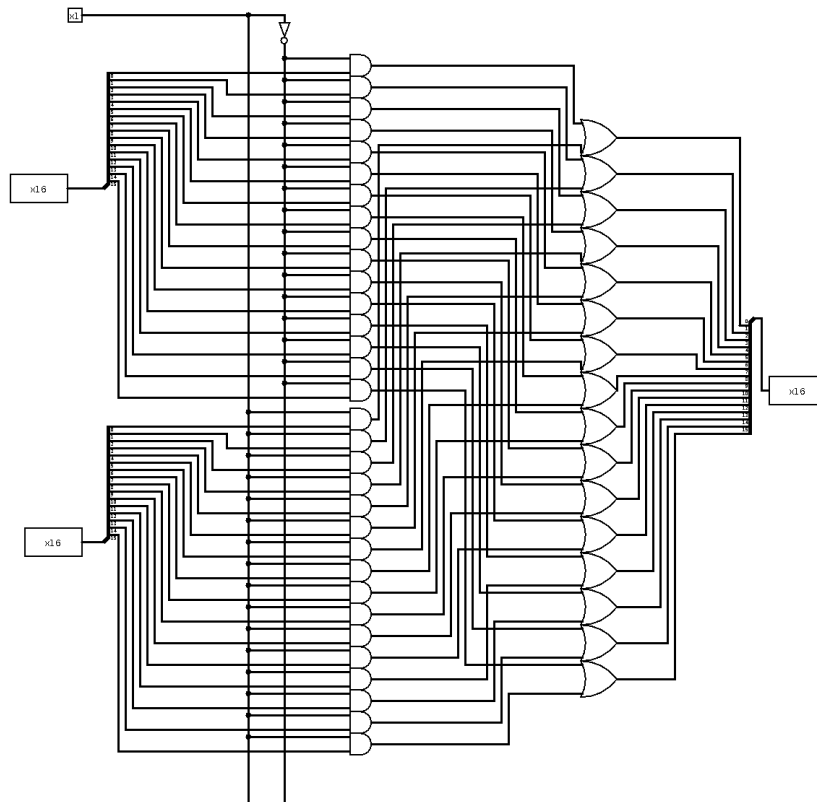


Fig. 4. 16-Bit Multiplexer Circuit (mux16.png) — 16-bit 2-to-1 MUX built from primitive gates in Logisim-Evolution.

The ALU_cricuits.circ file defines the main circuit named "main" and uses the standard Logisim library set: #Wiring (lib 0), #Gates (lib 1), #Plexers (lib 2), #Arithmetic (lib 3), #Memory (lib 4), #I/O (lib 5), and #Base (lib 6). The simulation parameters include a gate undefined behavior set to "ignore," a simulation limit of 1000 ticks, and simulation randomization disabled (simrand=0).

IV. Central Processing Unit (CPU)

The CPU is implemented in CPU.circ (1,192 lines of Logisim-Evolution XML, version 3.8.0). It is the most complex circuit in the project, integrating the ALU, A-register, D-register, program counter (PC), and instruction decoder into a unified execution engine. Figure 5 shows the CPU circuit in standard view and Figure 6 shows the annotated color-coded version.

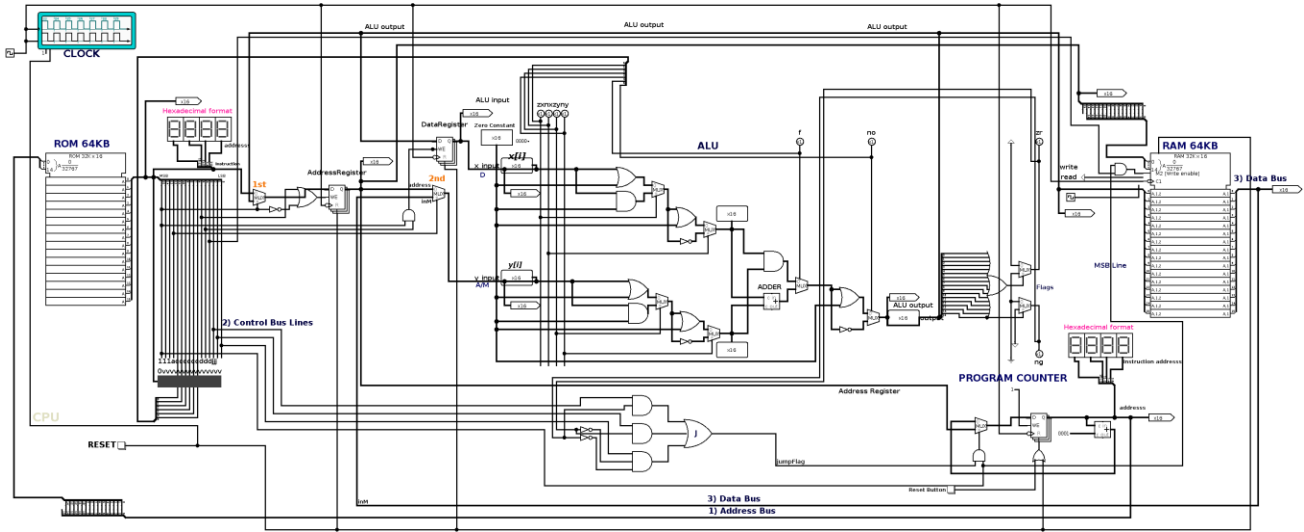


Fig. 5. CPU Circuit (CPU.png) — the complete 16-bit CPU design in Logisim-Evolution, showing all internal buses, multiplexers, the ALU, and register paths.

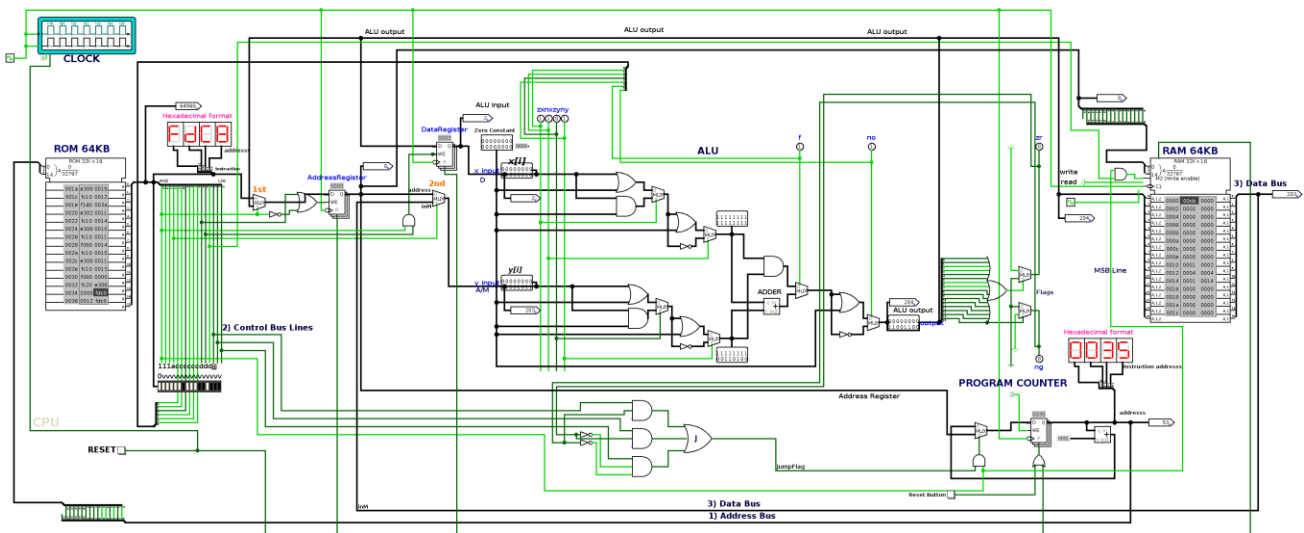


Fig. 6. CPU Circuit — Color-Coded (Colored.png) — the same CPU circuit with color-coded signal paths for clarity in debugging and documentation.

IV-A. CPU Subcomponents

The CPU.circ file uses Logisim-Evolution v3.8.0 libraries including additional TTL (lib 6), TCL (lib 7), BFH-Praktika (lib 9), Input/Output-Extra (lib 10), and Soc (lib 11) libraries beyond the standard set. The pin appearance is set to "classic" style. Key CPU subcomponents include:

- **Instruction Decoder:** Parses the 16-bit instruction word. If bit 15 is 0, the instruction is an A-instruction and the 15-bit value is loaded into the A-register. If bit 15 is 1, the instruction is a C-instruction and the decoder extracts the a-bit, cccccc, ddd, and jjj fields.

- **A-Register:** A 16-bit register that can be loaded from either the ALU output (via C-instruction destination) or directly from the instruction word (A-instruction). It drives both the ALU (when $a=0$) and the memory address bus.
- **D-Register:** A 16-bit register exclusively loaded from the ALU output when the destination field encodes D or includes D.
- **ALU:** Receives D and either A or M as inputs, performs the computation specified by the 6-bit comp field, and outputs a 16-bit result plus zero (zr) and negative (ng) status flags.
- **Program Counter (PC):** A 16-bit counter that normally increments by 1 each clock cycle. Jump logic overrides the PC to load the value of the A-register when the jump condition is satisfied.
- **Jump Logic:** Evaluates the `jjj` field against the ALU's zr and ng output flags to determine whether a jump should occur.

IV-B. Jump Instruction Encoding

The jump field (`jjj` bits 2:0 of C-instruction) is decoded as follows, directly from the Assembler's `ControlBits.jumper` array:

Mnemonic	jjj	Jump Condition
NULL	000	No jump (fall through to next instruction)
JGT	001	Jump if ALU output > 0 (not zero and not negative)
JEQ	010	Jump if ALU output $= 0$ (zero flag set)
JGE	011	Jump if ALU output ≥ 0 (not negative)
JLT	100	Jump if ALU output < 0 (negative flag set)
JNE	101	Jump if ALU output $\neq 0$ (zero flag not set)
JLE	110	Jump if ALU output ≤ 0 (zero or negative)
JMP	111	Unconditional jump to address in A-register

IV-C. Destination Field Encoding

The destination field (`ddd` bits 5:3) specifies which register(s) receive the ALU output:

Mnemonic	ddd	Destination
NULL	000	No write — result is discarded
M	001	Write to Memory[A]
D	010	Write to D-register
DM	011	Write to D-register and Memory[A]
A	100	Write to A-register
AM	101	Write to A-register and Memory[A]
AD	110	Write to A-register and D-register
ADM	111	Write to A-register, D-register, and Memory[A]

IV-D. Karnaugh Map — Jump Logic

The project includes `kmapJUMPsolution.pdf` in the CPU directory, which documents the Karnaugh map used to minimize the jump logic combinational circuit. The Karnaugh map takes the three `jjj` bits and the two ALU status flags (zr: zero, ng: negative) as inputs, and outputs a single JUMP enable signal. The minimized Boolean expression determines when the program counter should be loaded with the A-register value versus simply incrementing. This ensures correct implementation of all eight jump conditions using a minimal gate network.

IV-E. Hardware Debugger

The project includes a photograph of a hardware debugger (HardwareDebugger.jpeg, 720x1280 pixels) used during development for testing and validating the CPU behavior. This physical debugging instrument was employed to inspect signal states during live circuit execution, confirming the correctness of instruction decoding and ALU computations.

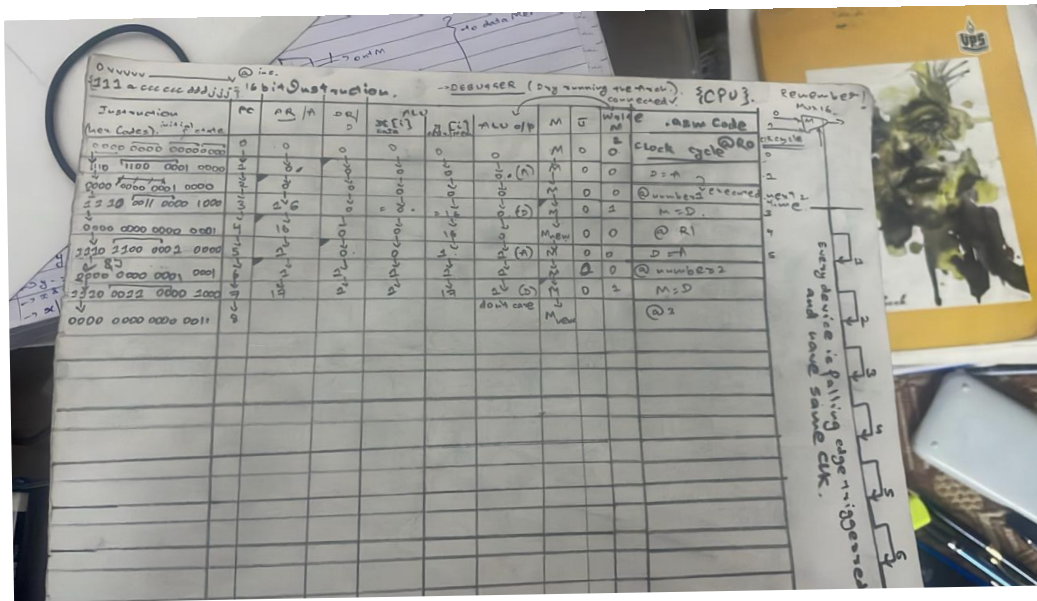


Fig. 7. Hardware Debugger photograph (HardwareDebugger.jpeg) — used during CPU development to inspect signal states and validate circuit behavior during live simulation.

V. Memory Architecture

The memory subsystem is one of the richest components of this project. It is implemented across multiple Logisim circuit files and thoroughly documented through images and experimental diagrams. The directory MEMORY/ contains the following files: 8RAM.circ, RAM32kB.circ, memory_fun.circ, and a rich collection of PNG images documenting memory design, address calculations, synchronization experiments, and register implementations.

V-A. 8-Word RAM Cell (8RAM.circ)

The 8RAM.circ file implements an 8-location, 16-bit-wide RAM module. This is the fundamental building block of the larger RAM hierarchy. Each cell stores one 16-bit word and is accessible via a 3-bit address bus. The design uses clocked D flip-flops (falling edge-triggered registers) for storage, enabling synchronous read and write operations. Figure 8 shows the 8RAM circuit.

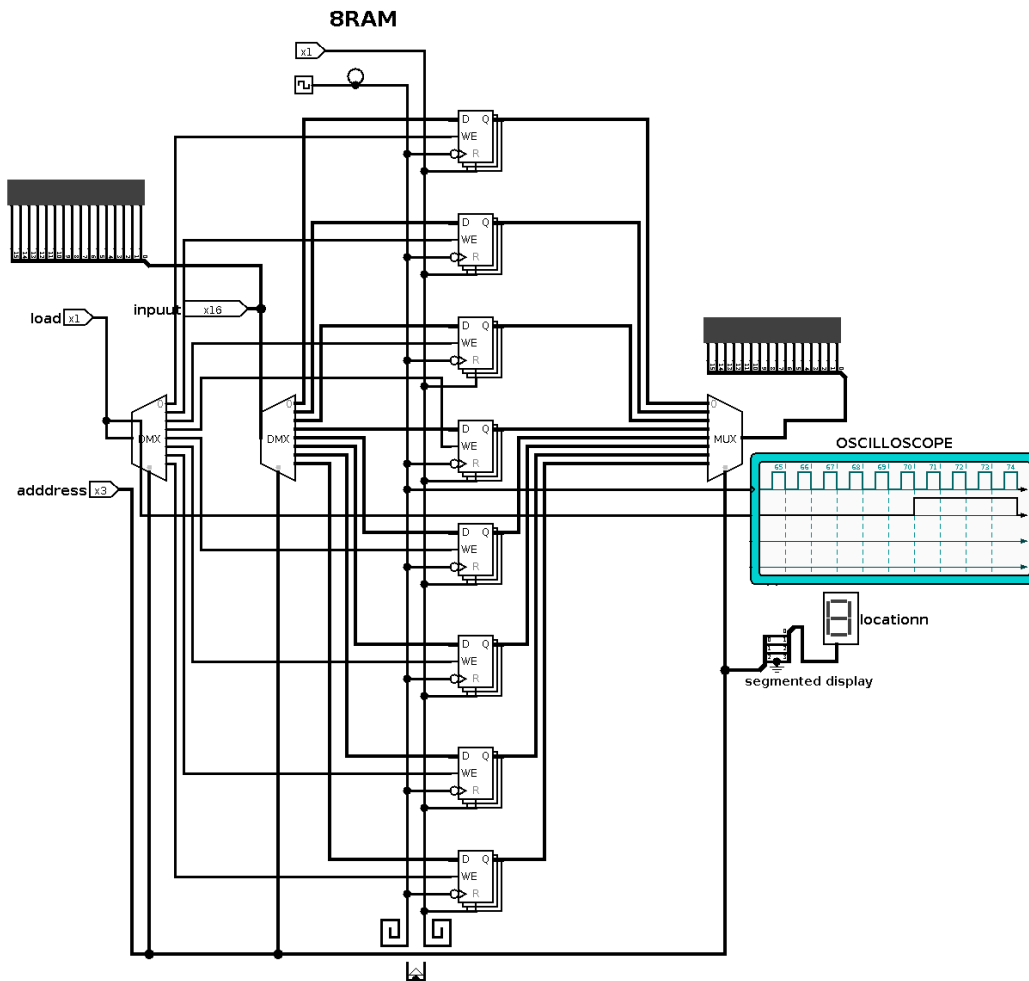


Fig. 8. 8-Word RAM Module (8RAM.png) — the foundational 8-location 16-bit RAM building block, showing address decoding, data bus, and register array.

RAM: Implementation

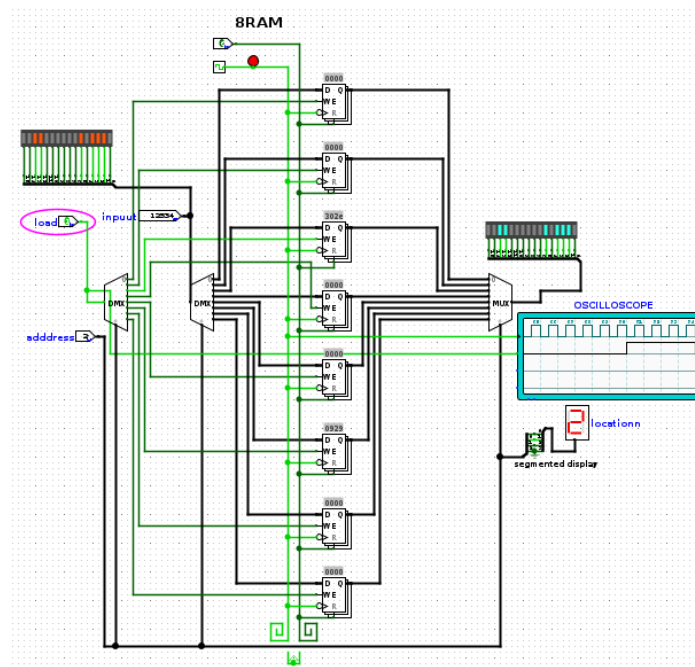
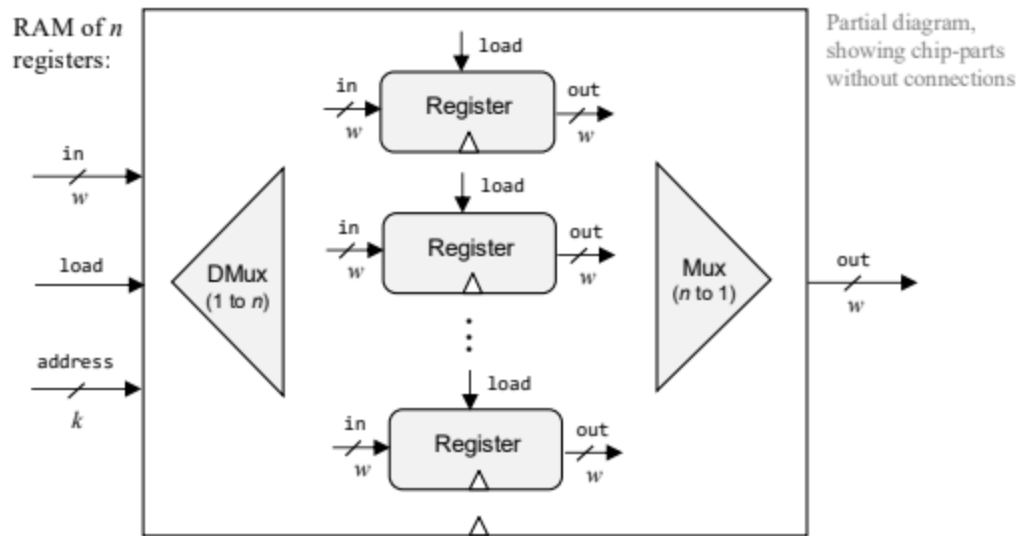


Fig. 9. 8-Word RAM — Color-Coded (8RAM_colored.png) — the same 8-word RAM with color-annotated signal paths for clarity.

V-B. 64KB RAM (RAM32kB.circ)

The RAM32kB.circ implements a 64-kilobyte (32,768 words x 16-bit) RAM module using a hierarchical design. The 15-bit address space ($2^{15} = 32,768$ locations) is decoded into a tree of smaller RAM blocks, ultimately reaching the 8RAM primitive cells. Figure 10 shows the 32KB RAM design (upgraded to 64KB) and Figure 11 provides the address calculation diagram.

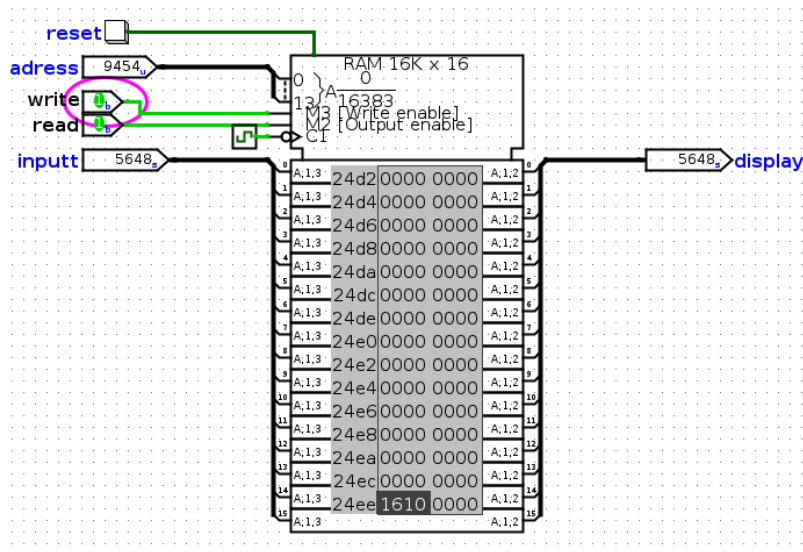


Fig. 10. 32KB RAM Architecture (RAM(32kBs).png) — hierarchical RAM design showing the memory block tree for 32,768 16-bit word storage.

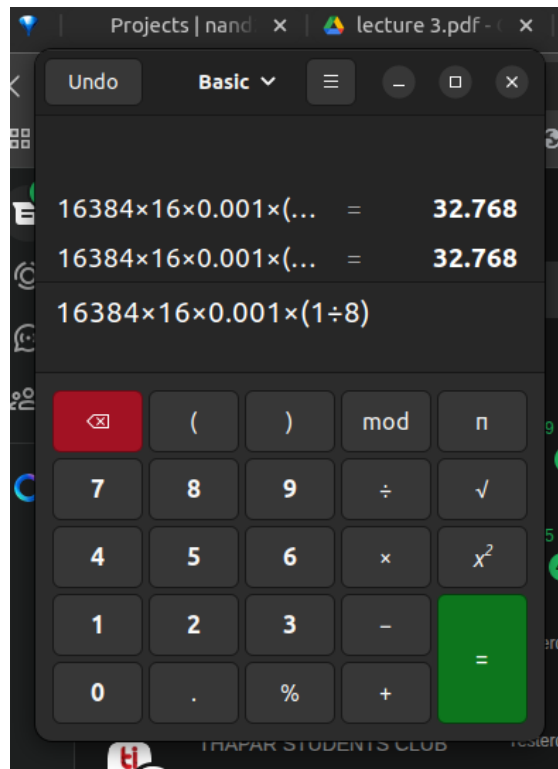


Fig. 11. RAM Capacity Calculation (RAM_calculation.png) — design worksheet showing address bit allocation and capacity calculation for the 32KB RAM.

The RAM32kB.circ uses Logisim-Evolution v3.8.0 with the same library set as the CPU. The simulation parameters match those of other circuits (simlimit=1000, simrand=0, gateUndefined=ignore).

V-E. Singular Register

A standalone 16-bit D flip-flop register design is documented in `singular_register.png`, serving as the atomic storage element from which all larger memory structures are composed. This register captures a 16-bit input value on the falling clock edge and holds it until the next write operation. Figure 14 shows this fundamental register circuit.

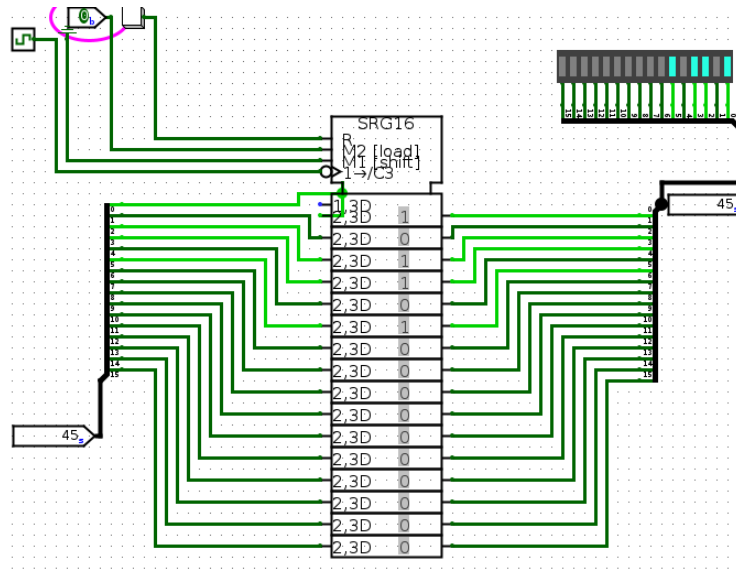


Fig. 14. Singular 16-Bit Register (`singular_register.png`) — the fundamental clocked storage element used as the building block for all RAM and CPU registers.

V-F. Memory Address Space and Framebuffer

Figure 15 shows the 16,384-location (32KB) view of the memory address space, documenting the base address of the screen framebuffer (address 16384 = 0x4000). This is the critical mapping that allows assembly programs to write to the screen by simply writing to memory addresses starting at 16384.

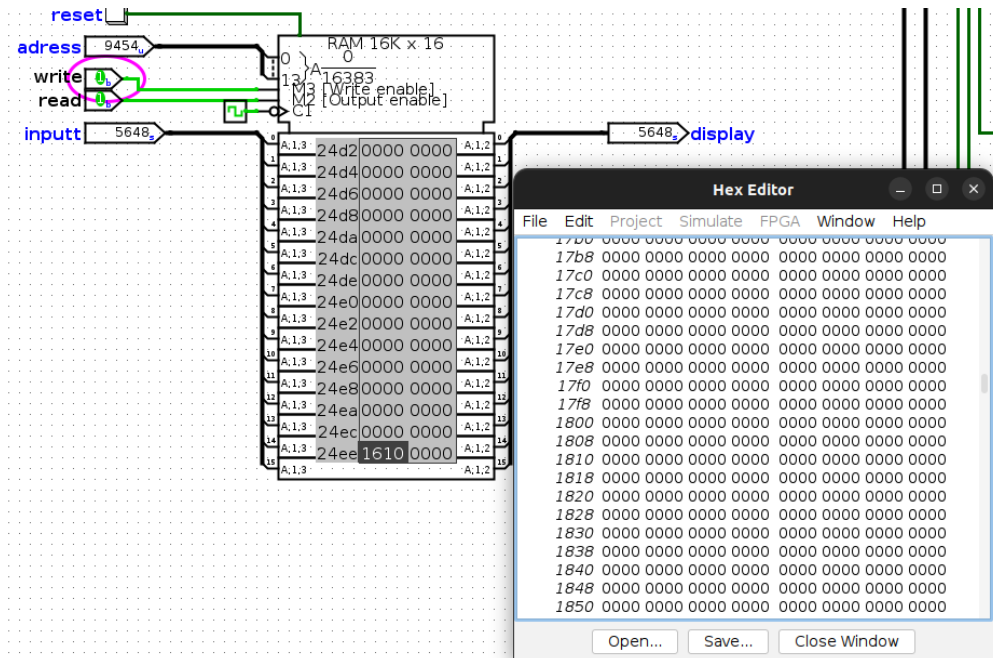


Fig. 15. Memory Address Mapping (`16384-locations.png`) — annotated view of the 16KB memory address space showing the screen framebuffer base address at 0x4000 (16384).

The total memory architecture of the system can be summarized as follows: the ROM (instruction memory) holds up to 64K 16-bit instructions loaded from the .hack or hex.txt output of the assembler. The RAM (data memory) provides 64K words of read/write storage for program variables. The screen framebuffer occupies 8K words of the RAM address space (4096 words $\times$ 16 bits/word = 65,536 bits = 64 x 64 pixels in 4096 words in RGB555 extended mode). The keyboard register at address 20481(as configured in the assembler) is a read-only memory-mapped register reflecting the ASCII code of the currently pressed key.

VI. Java Assembler — Full Implementation

The assembler is a complete two-pass assembly language translator implemented in Java. It reads assembly source code from `Code.asm`, performs symbol resolution and binary encoding, and outputs 16-bit binary machine code to standard output, which is redirected to `output.hack`. A companion `HexaDecimalConverter.java` then converts the binary output to hexadecimal format for loading into Logisim ROM components. A `TestScript.java` provides automated regression testing against a reference output. Together, these three Java programs form a complete assembler toolchain.

VI-A. Assembler.java — Architecture

The `Assembler` class contains two top-level data structures: a flat `ArrayList<String>` for the assembly source, and an `ArrayList<ArrayList<String>>` (the `translatedTable`) that returns both the translated code (index 0) and the variable/symbol table (index 1). The assembler pipeline consists of five functional stages:

1. Stage 1 — File Reading: The assembly source is read line-by-line from `Code.asm` using a `java.nio.file.Paths`-based `Scanner`.
2. Stage 2 — Refinement (`refiningFunction` / `RefineryFunction`): Removes blank lines, comment lines (lines starting with `/`), and inline whitespace. The refining function truncates each line at the first space character, effectively stripping inline comments and trailing whitespace. The developer notes in the source code acknowledge that empty line handling was non-obvious during development ("Revelation: Empty lines are handled automatically, idk man really idk it works!" and later "Update: This doesn't work 2 weeks later. Be careful y'all").
3. Stage 3 — Label Generation (`labelGenerationFunction`): A first pass over the refined code identifies all label declarations (lines beginning with `'()`). For each label, the function extracts the label name (`insideBracketFunction`), computes its ROM address (line index minus the count of label lines seen so far, since labels do not emit instructions), and stores it as `"LABELNAME;ADDRESS"` in the `loopVariables` list.
4. Stage 4 — Translation (`translationFunction`): The main translation pass processes each line of refined code. Label declarations are skipped (`continue`). A-instructions with numeric literals (`@0–@9...`) are converted to 16-bit binary using `binaryFunction(String)`. A-instructions with symbolic names are looked up in the symbol table; if not found, a new variable is created at the next available address (starting at 16, per convention). C-instructions are processed by the `ControlBits` inner class.
5. Stage 5 — Output (`printingFunction`): The translated binary strings are printed to standard output, one per line, for redirection to `output.hack`.

VI-B. Predefined Symbol Table

The `predefinedAddition()` method initializes the symbol table with all standard predefined symbols. The exact entries, extracted from the source code, are:

Symbol	Address	Purpose
R0–R15	0–15	General-purpose registers
SCREEN	16384	Base address of screen memory-mapped I/O
KBD	20481	Keyboard memory-mapped I/O address (project-specific value)
SP	0	Stack pointer (alias for R0)
LCL	1	Local variables base (alias for R1)
ARG	2	Arguments base (alias for R2)
THIS	3	This pointer (alias for R3)
THAT	4	That pointer (alias for R4)

VI-C. ControlBits Inner Class — C-Instruction Encoding

The ControlBits inner class handles all C-instruction encoding. It receives a raw instruction string and returns the 16-bit binary string "111accceccddjjj". The commandFunction() method identifies three cases: (1) the instruction contains both "=" and ";", meaning it has destination, computation, and jump fields; (2) the instruction contains ";" but not "=", meaning computation and jump only (NULL destination); and (3) the instruction contains "=" but not ";", meaning destination and computation only (NULL jump).

The computationFunction() handles the a-bit determination: if the computation string contains "M", the a-bit is set to "1" and the replaceFunction() substitutes all occurrences of M with A before table lookup. The inverseFunction() handles commutativity for binary operations — for example, A+D is treated equivalently to D+A by reversing the operand order and looking up D+A in the table.

The following excerpt from the Assembler source illustrates the ControlBits arrays as declared:

```
private String[] computation = {"0;101010", "1;111111", "-1;111010", "D;001100",
    "A;110000", "!D;001101", "!A;110001", "-D;001111", "-A;110011", "D+1;011111",
    "A+1;110111", "D-1;001110", "A-1;110010", "D+A;000010", "D-A;010011", "A-D;000111",
    "D&A;000000", "D|A;010101"};

private String[] destination =
    {"NULL;000", "M;001", "D;010", "DM;011", "A;100", "AM;101", "AD;110", "ADM;111"};

private String[] jumper =
    {"NULL;000", "JGT;001", "JEQ;010", "JGE;011", "JLT;100", "JNE;101", "JLE;110", "JMP;111"};
```

As you can see CSV format is very useful for data storage.

VI-D. Binary Conversion (binaryFunction)

Two overloaded versions of binaryFunction() are provided: one accepting a String (for @-literals, splitting on "@" to extract the number) and one accepting an int (for symbol resolution). Both implement the same radix-2 conversion algorithm: iterative division by 2 with remainder collection, followed by zero-padding to ensure a 16-bit output with a leading "0" (the A-instruction MSB). The algorithm is:

1. Extract integer value v from the string or use the int argument directly.
2. Compute remainder = $v \% 2$; quotient = $v / 2$.
3. Prepend remainder to the binary string; update $v = \text{quotient}$.
4. Repeat while quotient $\neq 0$, tracking digit count.
5. Pad with leading zeros to ensure 15 bits (plus the leading "0" for A-instruction).

VI-E. HexadecimalConverter.java

The HexadecimalConverter reads output.hack (the binary output of the assembler), converts each 16-bit binary string to a 4-digit hexadecimal string, and writes the result to hex.txt with a Logisim ROM image header ("v2.O raw"). The conversion processes each 16-bit binary string in four groups of 4 bits, computing the decimal value of each nibble and mapping values 10–15 to lowercase hex letters a–f. The output format is compatible with Logisim's ROM image loading. The hex.txt file begins with the header "v2.O raw" followed by one hexadecimal value per line.

VI-F. TestScript.java

The TestScript.java implements an automated regression test that compares output.hack (the assembler's output) line by line against HackAssemblerOutput.hack (the reference ground-truth binary). The script reads both files simultaneously using two Scanner instances, comparing each binary line string with String.equals(). Upon any mismatch, it records the line number and

sets MATCHING to false. At the end, it prints either "SUCCESS!" (all lines match) or "FAILURE!" with the first mismatching line number. The test script was used to validate the assembler against the standard Nand2Tetris assembler reference output, providing confidence in the binary encoding correctness.

The reference output file HackAssemblerOutput.hack contains 27 lines of 16-bit binary, representing a test assembly program. The first several lines of the reference output are: 0100000000000000, 1110110000010000, 0001000000000000, 1110000010010000, 0000000000010000, 1110001100001000, etc. — encoding a program with A-instructions and C-instructions for arithmetic computation.

VII. Assembly Language Programs

The `assembly_programs/` directory contains a rich collection of hand-written assembly language programs that demonstrate various aspects of the computer architecture. Programs range from fundamental arithmetic routines to large bitmap rendering programs with thousands of lines. This section documents each program in full detail.

SYNTAX:

`reg = {0 | 1 | -1}`

where `reg = {A | D | M}`

`reg1 = reg2`

where `reg1 = {A | D | M}`

`reg2 = {A | D | M}`

`reg1 = -reg2`

`reg1 = !reg2`

where `reg1 = {A | D | M}`

`reg2 = {A | D | M}`

`reg1 = reg2 + 1`

`reg1 = reg2 - 1`

where `reg1 = {A | D | M}`

`reg2 = {A | D | M}`

`reg1 = reg2 op reg3`

where `reg1 = {A | D | M}`

`reg2, reg3 = {A | D | M}`

`op = {+ | - | & | |}`

`[dest =] comp [; jump]`

`dest = {A | D | M | AD | AM | DM | ADM}`

`comp = any expression from above`

`jump = {JGT | JEQ | JGE | JLT | JNE | JLE | JMP}`

`@value`

where `value = {0..32767 | symbol | label}`

VII-A. fibonacci.asm — Fibonacci Sequence

Computes the first 14 Fibonacci numbers (1, 1, 2, 3, 5, 8, 13, 21, 34, 55...) using two memory variables (NUMBER1, NUMBER2) and a TEMPORARY swap variable. The algorithm initializes COUNTER=14 and both NUMBER1 and NUMBER2 to 1, then iterates a LOOP that: saves NUMBER2 to TEMPORARY, adds NUMBER1 to NUMBER2, saves the

new value to R2, copies TEMPORARY to NUMBER1, decrements COUNTER, and checks if $\text{COUNTER}-1-1 = 0$ (i.e., $\text{COUNTER} = 2$) to exit. The termination condition uses two successive $D=D-1$ operations before the JEQ test.

VII-B. MAX.asm — Maximum of Two Values

Finds the maximum of two numbers stored in R0 (initialized to 21) and R1 (initialized to 32). The algorithm computes $R0-R1$, stores it in R2, and jumps to FIRST if the result is ≥ 0 (R0 is the larger) or SECOND if < 0 (R1 is larger). The winning value is stored in R2 and the program enters an infinite END loop. This program demonstrates the JGE and JLT conditional jump instructions in their most fundamental use case.

VII-C. keyboard.asm — Keyboard Polling

A minimal keyboard polling loop that continuously reads the KBD memory-mapped register and compares it to ASCII value 65 (the letter "A"). If the value equals 65, the program jumps to END (an infinite loop), effectively halting when the key "A" is pressed. This demonstrates memory-mapped I/O access using the @KBD symbolic address, which resolves to 20481 in the assembler's predefined table. The program illustrates how hardware keyboard input is read entirely through memory access in this architecture.

VII-D. bitmap.asm — Full Screen Fill

Fills the entire screen framebuffer (8192 words) with all-ones ($-1 = 0xFFFF$ = all pixels on). The program computes $\text{LIMIT} = \text{SCREEN} + 8192 = 24576$, initializes $\text{INCREMENT} = \text{SCREEN} = 16384$, then enters a LOOP that: checks if $\text{INCREMENT} = \text{LIMIT}$ (exit condition), writes -1 to $\text{Memory}[\text{INCREMENT}]$, increments INCREMENT, and repeats. This creates a fully black screen in the 1-bit monochrome framebuffer model. The program demonstrates pointer-based sequential memory writes using the A=M pattern.

VII-E. RGB.asm — Colored Framebuffer Fill

Similar to bitmap.asm but fills only the first half of the framebuffer (4096 words = half screen) with the RGB555 color value 18165 (0x46F5 in hexadecimal, a specific shade). The program uses $\text{LIMIT} = \text{SCREEN} + 4096$ and writes the constant 18165 to each word. This demonstrates how a custom RGB color value (packed in the 555 format used by the project's extended display mode) is written to the screen memory. The value 18165 corresponds to a color with R=8 (bits 14:10), G=55 (bits 9:5), B=21 (bits 4:0) in the RGB555 packing.

VII-F. pointer_array.asm — Uniform Array Initialization

Initializes an array of 5 elements at memory addresses starting at R0 (address 300) with the uniform value 497. Uses a COUNTER variable and a UNIFORM_VALUE_INSERTION_LOOP to insert exactly 5 copies of 497 into consecutive memory locations. After insertion, R2 is restored to the original start address (300) and R0 is reset. This program demonstrates pointer-based array manipulation, a critical technique for higher-level data structure implementation.

VII-G. fibonacii_pointer_array_sum.asm — Fibonacci Sum with Array Storage

A more advanced program that computes Fibonacci numbers, stores each value in a pointer-addressed array (starting at address 202, using R0 as a moving pointer), and accumulates their sum in a SUM variable. The program uses 20 iterations ($\text{LIMITER}=20$), initializes the array manually ($@200\ M=0$, $@201\ M=1$, $@202\ D=A$, $R0=D$), and combines Fibonacci computation with array-indexed storage. After the loop, the final SUM is written to the last array position. This program demonstrates interleaved arithmetic computation and memory storage patterns.

VII-H. iterative_addition.asm — Sum of 1 to 100

Computes the sum of integers from 1 to 100 and stores the result in R1. Initializes $\text{COUNTER}=100$, $R0=1$, $\text{SUM}=1$, then loops decrementing COUNTER and adding an incrementing R0 to SUM until COUNTER reaches 1. The result (5050, the sum of $1+2+\dots+100$) is transferred to R1. This is a classic iterative summation demonstrating loop control with decremented counters.

VII-I. rom_execution.asm — Iterative Self-Addition

Demonstrates an unusual iterative self-doubling pattern. Loads R0 with 31, sets R1=2, then repeatedly adds R0 to itself R1 times (using ADDITION_LOOP that subtracts 1 from R1 each iteration until R1 hits 0). This explores how multiplication by small constants can be implemented via repeated addition in a system with no hardware multiplier.

VII-J. Soldier.asm — Pixel Art Bitmap (5208 Lines)

A large, auto-generated assembly program (5,208 lines) that renders a complex pixel art image of a soldier figure onto the RGB555 framebuffer. The program groups pixels by color value, loading each unique RGB555 color into D once and then writing it to all memory addresses corresponding to that color's pixel positions. The program begins with: "@32767 D=A" (loading 0x7FFF, white) and writes it to addresses 16399–16410 (top row pixels), followed by other color groups. This demonstrates how the BitmapEditor.html tool's "Copy ASM" function outputs optimized assembly by grouping same-colored pixels together to minimize @-instruction overhead.

VII-K. 3DShapes.asm — 3D Shape Rendering (2768 Lines)

A 2,768-line assembly program that renders 3D geometric shapes (cubes, spheres, or similar) on the RGB555 framebuffer. Like Soldier.asm, it is generated by the Bitmap Editor and groups pixels by color. The opening comment identifies it as "RGB555 Bitmap — 1361 pixel(s)" with a framebuffer base of 16384. Color groups include 0x0137 (14 pixels, a dark blue-ish tone) and 0x6042 (50 pixels, a brownish-green), each written to specific framebuffer addresses. This program demonstrates how complex 3D geometric imagery can be pre-rendered and stored as assembly language for playback on the custom computer.

VIII-A. Screen Subsystem

The Backup/Screen.circ file (57,008 bytes) provides a backup copy of the screen circuit, and the project acknowledges the use of an external Logisim keyboard/screen library from <https://github.com/dhnam/n2tLogisimIO> for the hardware I/O simulation in Logisim.

VIII-B. Keyboard Subsystem

The keyboard is mapped to a single 16-bit read-only register at address 24576 (KBD). When a key is pressed, the hardware (or Logisim keyboard simulation library) places the ASCII code of the pressed key in this register. When no key is pressed, the register contains 0. The assembler's predefined table maps KBD to 20481, which is a project-specific configuration.

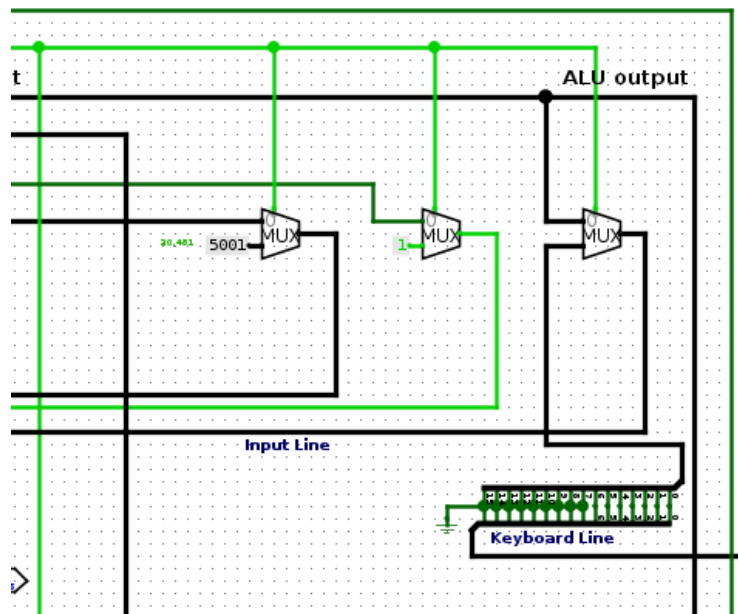


Fig. 17. Keyboard Subsystem (Keyboard.png) — Logisim-Evolution view of the keyboard memory-mapped I/O circuit, integrated with the n2tLogisimIO library.

The keyboard.asm assembly program demonstrates the polling pattern: the program continuously reads @KBD into D and subtracts 65 (ASCII "A"), jumping to END when D equals 0. This busy-wait polling loop is the simplest possible keyboard driver and is appropriate for the architecture's current capability level.

The Backup/Keyboard.circ file (64,005 bytes) provides a backup of the keyboard circuit implementation. The project notes in references.txt point to the ACM digital library paper at <https://dl.acm.org/doi/epdf/10.1145/3587102.3588858> as a relevant research reference for the screen/keyboard I/O approach, and to the dhnam/n2tLogisimIO GitHub repository for the Logisim library implementation.

IX. RGB555 Bitmap Editor Tool (BitmapEditor.html)

BitmapEditor.html is a 698-line, fully self-contained single-file web application implementing an interactive RGB555 pixel art editor. It runs entirely in the browser with no external dependencies and is hosted on GitHub Pages at <https://modern-day-computing-system.github.io/PROJECT/>. The editor provides a visual interface for designing pixel art in the RGB555 color format used by the custom computer's extended display mode, and generates valid assembly language code that can be directly loaded into the assembler.

IX-A. User Interface Architecture

The editor uses a monospace font throughout and implements a CSS custom property (CSS variable) theming system with both light and dark modes. The layout consists of four main panels:

- Left panel (#left): Contains the drawing canvas, coordinate display, and manual address input.
- Right panel (#right): Contains color picker, tool selection, character placement, shape tools, display options, and action buttons.
- ASM output panel (#asm-box): Displays the generated assembly code in real-time as pixels are drawn.
- Graphics preview panel (#graphics-panel): Shows an animation canvas for preview rendering.

IX-B. Drawing Canvas

The canvas is a 64×64 pixel grid rendered as an HTML div grid (#grid), with each cell being an 8×8 CSS pixel square, producing a 512×512 pixel visual canvas. An SVG overlay (#overlay) handles hover effects and selection highlighting. Row and column number labels are displayed alongside the grid for coordinate reference. The grid supports optional grid line display (toggled by the "Grid Lines" button using the "segmented" CSS class).

IX-C. Drawing Tools

The editor provides the following tools activated by toolbar buttons:

Tool	Description
 Draw	Freehand pixel painting with configurable brush size (range input, 1–8 pixels)
 Erase	Erase pixels by setting them to the background color (empty)
 Fill	Flood-fill (paint bucket) from a clicked pixel, changing all connected same-color pixels
 Place	Character/stamp mode — places a predefined character glyph at the clicked position using the current color
 Rect	Rectangle shape drawing in two-click (anchor + drag) placement mode
 Circle	Circle shape drawing
 Triangle	Triangle shape drawing
 Line	Line drawing using Bresenham's algorithm or similar
 Undo	Undo last drawing action (implemented in the most recent git commit: "added undo button")
 Rotate	Rotates the current shape before placement, with angle shown in degrees
 Blink Pixel	Blinks the pixel at a manually entered memory address for hardware testing

IX-D. Color System — RGB555

The editor uses the RGB555 color format, in which each color is encoded as a 15-bit value: bits 14:10 = Red (0–31), bits 9:5 = Green (0–31), bits 4:0 = Blue (0–31). This matches the extended display mode of the custom computer. The color picker uses an `HTML input[type=color]` element and displays the selected color's RGB555 encoding, decimal value, and hex value in the

#color-info div. The displayed color info includes the format: RGB values in 0–255 space mapped to 0–31 (5-bit each), the packed 15-bit decimal value, and the hex representation.

IX-E. Assembly Code Generation

The central feature of the BitmapEditor is its real-time assembly code generation. As the user draws pixels, the #asm-out panel updates with valid assembly language that, when loaded into the assembler, will reproduce the drawing on the screen of the custom computer. The generated code follows the format:

```
// RGB555 Bitmap - N pixel(s)
// Framebuffer base: 16384

// Color 0XXXXX - M px
@<rgb555_decimal_value>
D=A
@<framebuffer_address>
M=D
@<framebuffer_address+1>
M=D
// ... (repeat for each pixel of this color)
```

The code generation optimizes by grouping pixels of the same color together, loading the color value into D once and then writing it to all matching framebuffer addresses sequentially. This reduces instruction count by avoiding redundant @<color> and D=A instruction pairs. The "#asm-stats" div displays statistics such as total pixels, unique colors, and instruction count. The "📄 Copy ASM" button copies the generated assembly to the clipboard for pasting into Code.asm.

X. Project Infrastructure, Git History, and Build System

X-A. Git Repository and Version History

The project is managed with Git version control. The repository has two active branches: "main" (primary development) and "BitmapEditor-pages" (GitHub Pages deployment of the editor). The commit history spans from March 15, 2026 to May 19–20, 2026, with the following key commits documented in the git log:

Commit	Description
138e2c4	Added undo button — most recent commit, adding undo functionality to BitmapEditor
439d7a3	Add you submissions here cuties — community submission folder opened to contributions
93342f5	Add your submissions here — preparatory commit for community features
bdf5c3a	Add your submission folder to this — folder infrastructure for submissions
e414059	RGB --> RGB555 format now — major upgrade to 15-bit color display mode
1a2f912	website bitmap — GitHub Pages deployment of BitmapEditor
f6c0800	Include MIT License file — licensing added
a052c6e	readme nice — README formatting improvements
ffd7695	Keyboard and Screen added — I/O subsystems integrated
ef6c556	updated readme — documentation update
273ea23	references added — external references file created
b267ce8	just some pictures debugger — hardware debugger photos added
6d60141	CPU done — CPU circuit completed

The .git/MERGE_MSGHello file contains a merge message from May 5, 2026, indicating a branch merge was performed during development. The repository also maintains a stash (refs/stash) from the initial development phase (March 15, 2026).

X-B. Build and Run Instructions

The project uses the following build workflow:

1. Compiling the Assembler

```
cd Assembler
javac Assembler.java
javac HexadecimalConverter.java
javac TestScript.java
```

2. Assembling a Program

Place assembly code in Code.asm, then:

```
java Assembler > output.hack
```

3. Converting to Hexadecimal for Logisim

```
java HexadecimalConverter
```

Output is written to hex.txt with the Logisim ROM image header "v2.O raw" followed by one hex value per line. Load hex.txt into the ROM component in Logisim-Evolution via the "Load Image" option.

4. Running the Test Script

```
java TestScript
```

Compares output.hack against HackAssemblerOutput.hack and reports SUCCESS or FAILURE with the first mismatching line number.

5. Opening Circuits in Logisim-Evolution

```
logisim-evolution CPU/CPU.circ
logisim-evolution ALU/adder.circ
logisim-evolution MEMORY/RAM32kB.circ
logisim-evolution COMPUTER/Model1.circ
```

X-C. Course Work — Academic Poster

The Course Work/ directory contains Poster.png (2245x1587 pixels, 720,854 bytes), an academic poster created for a university course submission. This poster summarizes the project's architecture, components, and achievements in a format suitable for academic presentation. The poster demonstrates the project's educational context and its relationship to formal computer architecture coursework.

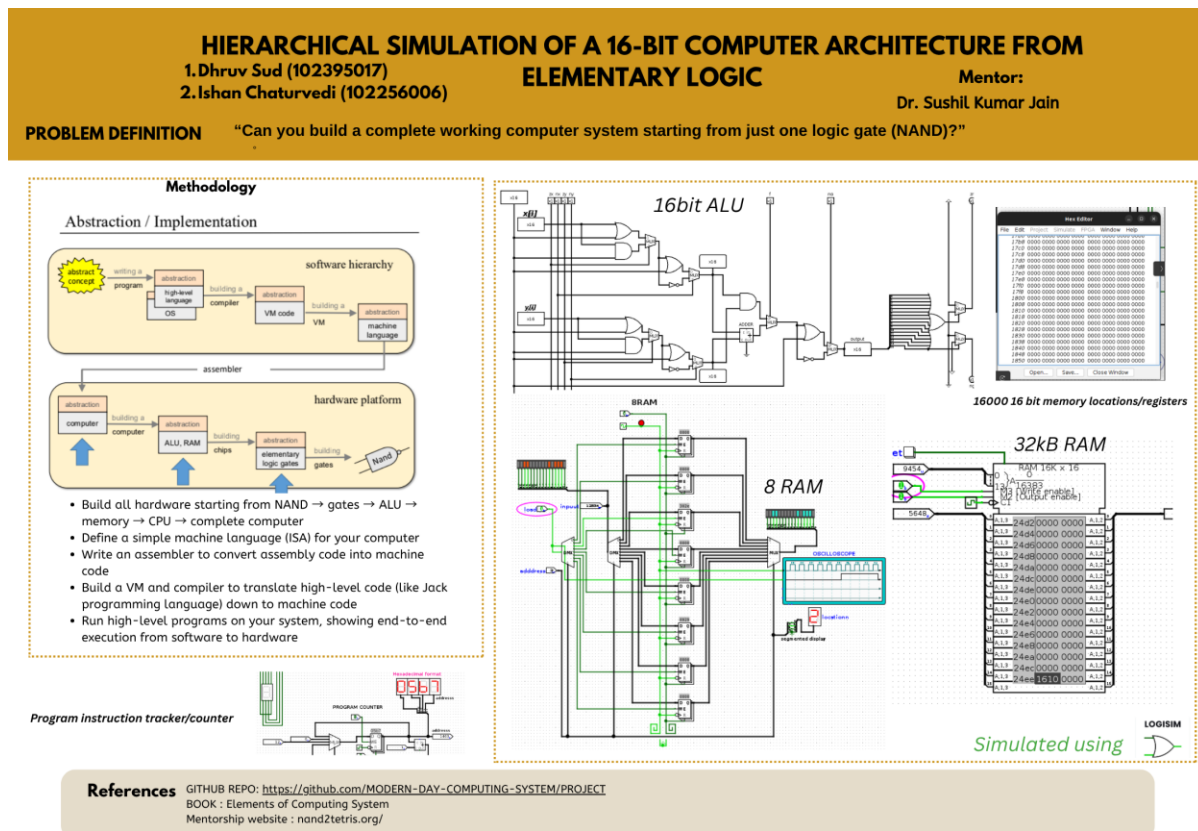


Fig. 18. Academic Course Poster (Poster.png) — the project's academic poster summarizing the system architecture, key components, and learning outcomes for course submission.

X-D. Reference Books

The Books/ directory contains two primary reference texts included in the repository (totaling approximately 35MB):

- "The Elements of Computing Systems: Building a Modern Computer from First Principles" by Noam Nisan and Shimon Schocken (Nand2Tetris) — the primary inspiration for the instruction set architecture, assembler design, and overall project structure.
- "Digital Computer Electronics" by Albert Paul Malvino — a classic reference for digital electronics and computer organization, providing foundational theory for the gate-level circuit design.

XI. Conclusion and Future Work

The Modern Day Computing System project successfully demonstrates the construction of a complete 16-bit computing platform from first principles. Starting from primitive logic gates simulated in Logisim-Evolution, the project implements a full Von Neumann architecture including a functional ALU with 18 operations, a CPU with instruction decoding and jump logic, a hierarchical 32KB RAM, ROM-based instruction memory, memory-mapped screen and keyboard I/O, and a fully working Java-based assembler toolchain. The assembly language programs range from simple arithmetic to thousands of lines of pixel art rendering code, demonstrating the breadth of the system's capabilities.

The BitmapEditor.html web tool represents a particularly innovative contribution: it bridges the gap between visual creativity and low-level systems programming by allowing users to design pixel art graphically and instantly obtain the assembly code needed to display it on the custom hardware. The collaborative PixelCommunitySubmissions feature further demonstrates the educational and community-building potential of the project.

The project is explicitly marked as a "Work In Progress" with a clear roadmap for future development. Planned improvements include:

- VGA-style video output — replacing the simple memory-mapped display with a VGA signal generator for standard monitor connectivity.
- Faster system execution — optimizing the Logisim simulation speed and potentially implementing the design on FPGA hardware.
- Hardware debugger improvements — building a more comprehensive debugging interface integrated with the circuit simulation.
- Keyboard buffering — implementing a hardware keyboard buffer to prevent key loss during rapid typing.
- Custom instruction extensions — extending the instruction set with new operations such as multiplication, division, or shift instructions.
- Simple operating system experiments — implementing a minimal OS kernel on top of the hardware, exploring process management and memory allocation.
- Compiler backend exploration — implementing a compiler that targets the custom assembly language.
- Graphics acceleration concepts — exploring hardware-assisted drawing primitives for improved display performance.

This project achieves its core educational objective: to make the inner workings of a computer tangible, observable, and personally constructed. Every component has been built, tested, and documented, providing the author with deep insight into computer architecture at a level that no amount of high-level programming can provide. The project is offered to the community under the MIT License as both an educational resource and a foundation for future exploration.

XII. References

- [1] N. Nisan and S. Schocken, The Elements of Computing Systems: Building a Modern Computer from First Principles. Cambridge, MA: MIT Press, 2005. [Included in PROJECT/Books/]
- [2] A. P. Malvino, Digital Computer Electronics. New York: McGraw-Hill, 1993. [Included in PROJECT/Books/]
- [3] "ACM Digital Library Paper," available at: <https://dl.acm.org/doi/epdf/10.1145/3587102.3588858> [Referenced in references.txt]
- [4] D. Nam, "n2tLogisimIO: Nand2Tetris Screen and Keyboard for Logisim," GitHub repository, available at: <https://github.com/dhnam/n2tLogisimIO> [Used for Logisim I/O library]
- [5] Logisim-Evolution Development Team, "Logisim-Evolution," version 3.8.0, GitHub repository, available at: <https://github.com/logisim-evolution/> [Circuit simulation tool used throughout project]
- [6] D. Sud, "MODERN DAY COMPUTING SYSTEM," GitHub repository, 2026, available at: <https://modern-day-computing-system.github.io/PROJECT/> [Primary project repository]
- [7] Oracle Corporation, "Java Development Kit (JDK)," available at: <https://www.oracle.com/java/> [Used for Assembler implementation]
- [8] MIT License, "Open Source Initiative," available at: <https://opensource.org/licenses/MIT> [License governing this project]